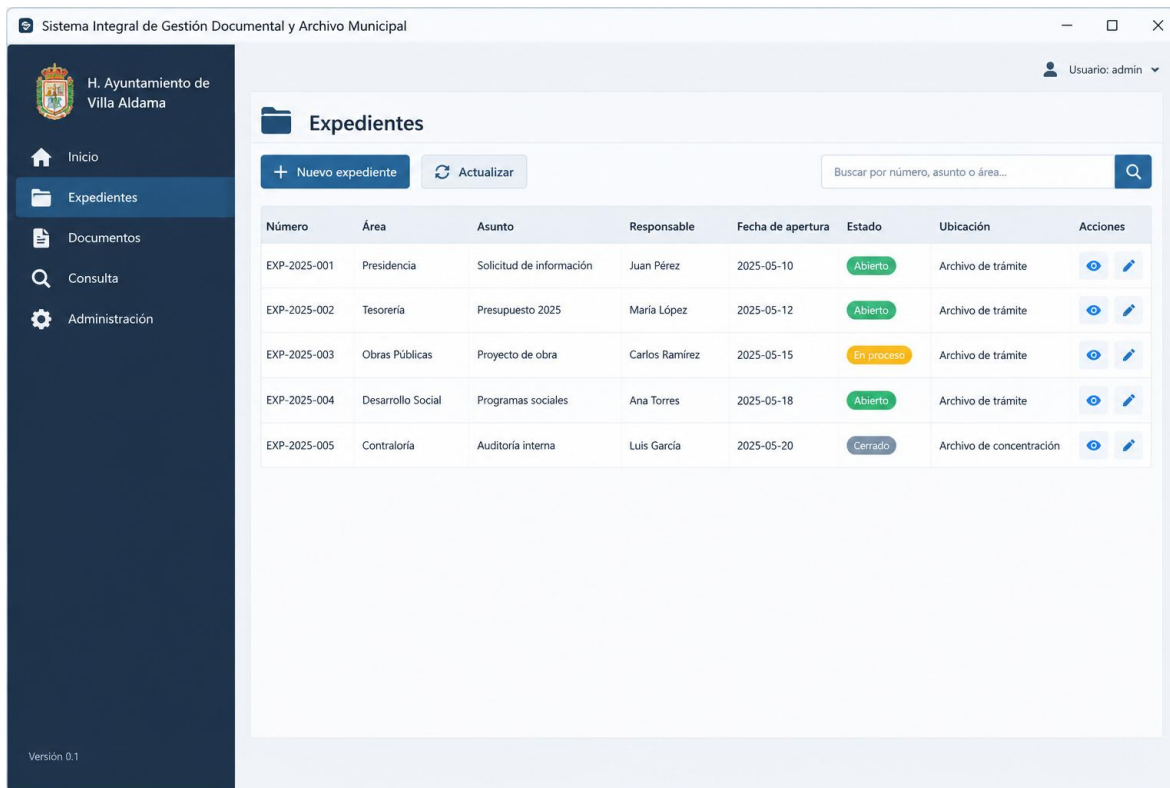


OBJETIVO



1. Nombre del incremento

E4 — Incremento 1: Registro y consulta inicial de expedientes

Proyecto

Sistema Integral de Gestión Documental y Archivo Municipal

Institución

H. Ayuntamiento de Villa Aldama, Veracruz

Objetivo

Construir la primera versión ejecutable del sistema, implementando una estructura inicial orientada a objetos, persistencia en base de datos y una interfaz gráfica funcional.

En este primer incremento no se pretende desarrollar todavía todo el sistema archivístico. La finalidad es construir una base sólida que posteriormente permita agregar usuarios,

documentos, clasificación archivística, transferencias, préstamos, auditoría, reportes y demás módulos.

La documentación del proyecto establece precisamente que la versión 1.0 debe servir como prototipo inicial y que las versiones posteriores deben ampliar las anteriores conservando la información y estructura existente.

2. ¿Qué vamos a conseguir?

Al terminar este incremento, el programa deberá poder:

1. Abrirse como aplicación de escritorio.
2. Mostrar el nombre del sistema.
3. Mostrar el Ayuntamiento.
4. Mostrar un menú lateral.
5. Mostrar el módulo **Expedientes**.
6. Mostrar una tabla de expedientes.
7. Registrar un expediente nuevo.
8. Guardarlo realmente en MySQL.
9. Actualizar la tabla.
10. Buscar por número, asunto o área.
11. Consultar los datos almacenados.
12. Mostrar el usuario activo.
13. Manejar estados como:
 - Abierto
 - En proceso
 - Cerrado
14. Mantener una estructura de clases, interfaces, DAO y servicios.

El expediente será la unidad principal de este primer flujo. La especificación del proyecto contempla para un expediente elementos como identificador, número de expediente, fondo, sección, serie, área productora, responsable, asunto, fechas, estado, ubicación y documentos asociados. En este primer incremento utilizaremos solamente una parte de esos campos, dejando preparada la estructura para ampliarlos posteriormente.

3. Resultado visual esperado

La primera pantalla deberá aproximarse al diseño generado:

- Barra lateral azul oscuro.
- Escudo/nombre del Ayuntamiento.
- Inicio.
- Expedientes.
- Documentos.
- Consulta.
- Administración.
- Usuario activo.
- Título "Expedientes".
- Botón "Nuevo expediente".
- Botón "Actualizar".
- Buscador.
- Tabla.
- Número de expediente.
- Área.
- Asunto.
- Responsable.
- Fecha de apertura.
- Estado.
- Ubicación.
- Acciones.

La interfaz no será solamente una imagen: todos los elementos principales deberán estar contruidos mediante código Java Swing.

4. Distribución del trabajo entre cuatro integrantes

INTEGRANTE 1 — Base de datos y conexión

Responsable de:

- Crear la base de datos.
- Crear tablas.
- Crear relaciones.
- Insertar datos iniciales.
- Configurar MySQL.
- Crear `ConexionBD.java`.
- Probar la conexión.
- Documentar la base de datos.

Entregable

Una base de datos funcionando y una clase Java capaz de conectarse a ella.

INTEGRANTE 2 — Programación orientada a objetos

Responsable de:

- Crear las clases modelo.
- Implementar encapsulamiento.
- Crear getters y setters.
- Crear interfaces.
- Crear DAO.
- Crear implementación DAO.
- Crear servicios.

Clases iniciales:

Expediente.java
Usuario.java
Area.java

ExpedienteDAO.java
ExpedienteDAOImpl.java

ExpedienteService.java

Entregable

La estructura lógica del sistema sin depender directamente de la interfaz.

INTEGRANTE 3 — Interfaz gráfica

Responsable de:

- Crear la ventana principal.
- Crear menú lateral.
- Definir colores.
- Definir posiciones.
- Crear tabla.
- Crear botones.
- Crear buscador.
- Crear formulario de nuevo expediente.
- Crear diseño visual.

Entregable

La interfaz funcionando visualmente.

INTEGRANTE 4 — Integración y pruebas

Responsable de:

- Integrar interfaz + clases + BD.
- Probar registro.
- Probar consulta.
- Probar búsqueda.
- Detectar errores.
- Verificar que los datos permanezcan después de cerrar el programa.
- Crear capturas de evidencia.
- Controlar Git.
- Preparar demostración.

Entregable

Aplicación ejecutable con el flujo completo.

5. Flujo que deberá demostrar el equipo

La demostración debe ser:

```
INICIO
↓
Sistema de Archivo Municipal
↓
Expedientes
↓
Nuevo expediente
↓
Capturar información
↓
Guardar
↓
MySQL
↓
Actualizar tabla
↓
Buscar expediente
↓
```

Consultar información

Este flujo es suficiente para demostrar una primera funcionalidad completa de registro y consulta.

6. Crear el proyecto Java

Se puede realizar en **Eclipse** o **NetBeans**.

Para mantenerlo sencillo, utilizaremos:

Java
Swing
MySQL
JDBC

No necesitamos todavía un framework web.

El proyecto puede llamarse:

SistemaArchivoMunicipal

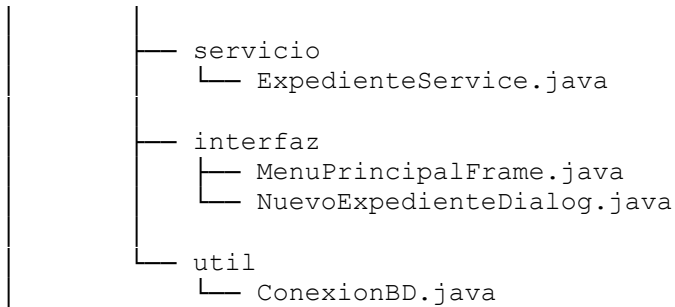
El paquete principal:

archivo.municipal

7. Estructura del proyecto

Crear:

```
SistemaArchivoMunicipal
├── src
│   └── archivo.municipal
│       ├── Main.java
│       ├── modelo
│       │   ├── Expediente.java
│       │   ├── Usuario.java
│       │   └── Area.java
│       └── dao
│           ├── ExpedienteDAO.java
│           └── ExpedienteDAOImpl.java
```



Esta separación es importante porque evita meter toda la programación en una sola ventana.

8. Crear la base de datos

En MySQL Workbench, phpMyAdmin o el cliente que utilicen, ejecutar:

```
CREATE DATABASE archivo_municipal;
```

```
USE archivo_municipal;
```

Crear las áreas:

```
CREATE TABLE areas (
    id_area INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    descripcion VARCHAR(255),
    estado BOOLEAN DEFAULT TRUE
);
```

Insertar algunas áreas de prueba:

```
INSERT INTO areas (nombre, descripcion) VALUES
('Presidencia', 'Presidencia Municipal'),
('Tesorería', 'Tesorería Municipal'),
('Obras Públicas', 'Dirección de Obras Públicas'),
('Desarrollo Social', 'Dirección de Desarrollo Social'),
('Contraloría', 'Órgano de Control Interno');
```

Crear usuarios:

```
CREATE TABLE usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    nombre_completo VARCHAR(150) NOT NULL,
    usuario VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    rol VARCHAR(50) NOT NULL,
    estado BOOLEAN DEFAULT TRUE
);
```

Agregar usuario inicial:

```
INSERT INTO usuarios
(nombre_completo, usuario, password, rol)
VALUES
('Administrador del Sistema', 'admin', 'admin123', 'Administrador');
```

Crear expedientes:

```
CREATE TABLE expedientes (
    id_expediente INT AUTO_INCREMENT PRIMARY KEY,
    numero_expediente VARCHAR(50) NOT NULL UNIQUE,
    id_area INT NOT NULL,
    asunto VARCHAR(255) NOT NULL,
    responsable VARCHAR(150) NOT NULL,
    fecha_apertura DATE NOT NULL,
    estado VARCHAR(30) NOT NULL,
    ubicacion VARCHAR(150) NOT NULL,

    CONSTRAINT fk_expediente_area
    FOREIGN KEY (id_area)
    REFERENCES areas(id_area)
);
```

Agregar algunos registros para que la interfaz no aparezca vacía:

```
INSERT INTO expedientes
(numero_expediente, id_area, asunto, responsable,
 fecha_apertura, estado, ubicacion)
VALUES
('EXP-2026-001', 1, 'Solicitud de información',
 'Administrador del Sistema', '2026-10-01',
 'Abierto', 'Archivo de trámite'),

('EXP-2026-002', 2, 'Presupuesto municipal',
 'Administrador del Sistema', '2026-10-02',
 'Abierto', 'Archivo de trámite'),

('EXP-2026-003', 3, 'Proyecto de obra',
 'Administrador del Sistema', '2026-10-03',
 'En proceso', 'Archivo de trámite');
```

9. Conexión a MySQL

Crear:

util/ConexionBD.java

Código:

```
package archivo.municipal.util;
```

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {

    private static final String URL =
        "jdbc:mysql://localhost:3306/archivo_municipal";

    private static final String USUARIO = "root";
    private static final String PASSWORD = "";

    public static Connection conectar() throws SQLException {
        return DriverManager.getConnection(
            URL,
            USUARIO,
            PASSWORD
        );
    }
}

```

Si MySQL tiene contraseña, colocarla:

```
private static final String PASSWORD = "TU_CONTRASEÑA";
```

También deben agregar al proyecto el:

```
mysql-connector-j
```

10. Crear la clase Expediente

Crear:

```
modelo/Expediente.java
```

Código:

```

package archivo.municipal.modelo;

import java.time.LocalDate;

public class Expediente {

    private int id;
    private String numeroExpediente;
    private int idArea;
    private String area;
    private String asunto;
    private String responsable;
    private LocalDate fechaApertura;
}

```

```
private String estado;
private String ubicacion;

public Expediente() {
}

public Expediente(String numeroExpediente,
                  int idArea,
                  String asunto,
                  String responsable,
                  LocalDate fechaApertura,
                  String estado,
                  String ubicacion) {

    this.numeroExpediente = numeroExpediente;
    this.idArea = idArea;
    this.asunto = asunto;
    this.responsable = responsable;
    this.fechaApertura = fechaApertura;
    this.estado = estado;
    this.ubicacion = ubicacion;
}

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getNumeroExpediente() {
    return numeroExpediente;
}

public void setNumeroExpediente(String numeroExpediente) {
    this.numeroExpediente = numeroExpediente;
}

public int getIdArea() {
    return idArea;
}

public void setIdArea(int idArea) {
    this.idArea = idArea;
}

public String getArea() {
    return area;
}

public void setArea(String area) {
    this.area = area;
}

public String getAsunto() {
    return asunto;
}
```

```

    }

    public void setAsunto(String asunto) {
        this.asunto = asunto;
    }

    public String getResponsable() {
        return responsable;
    }

    public void setResponsable(String responsable) {
        this.responsable = responsable;
    }

    public LocalDate getFechaApertura() {
        return fechaApertura;
    }

    public void setFechaApertura(LocalDate fechaApertura) {
        this.fechaApertura = fechaApertura;
    }

    public String getEstado() {
        return estado;
    }

    public void setEstado(String estado) {
        this.estado = estado;
    }

    public String getUbicacion() {
        return ubicacion;
    }

    public void setUbicacion(String ubicacion) {
        this.ubicacion = ubicacion;
    }
}

```

Aquí ya estamos demostrando **POO y encapsulamiento**, porque los atributos son privados y se accede a ellos mediante métodos.

11. Crear la interfaz DAO

Crear:

dao/ExpedienteDAO.java

Código:

```
package archivo.municipal.dao;
```

```
import archivo.municipal.modelo.Expediente;
import java.util.List;

public interface ExpedienteDAO {

    boolean guardar(Expediente expediente);

    List<Expediente> listar();

    List<Expediente> buscar(String texto);
}
```

Esta interfaz es importante para el requisito académico de trabajar con interfaces.

12. Implementar el DAO

Crear:

dao/ExpedienteDAOImpl.java

Código:

```
package archivo.municipal.dao;

import archivo.municipal.modelo.Expediente;
import archivo.municipal.util.ConexionBD;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import java.util.ArrayList;
import java.util.List;

public class ExpedienteDAOImpl implements ExpedienteDAO {

    @Override
    public boolean guardar(Expediente expediente) {

        String sql = ""
            INSERT INTO expedientes
            (numero_expediente, id_area, asunto,
             responsable, fecha_apertura, estado, ubicacion)
            VALUES (?, ?, ?, ?, ?, ?, ?)
            "";

        try (Connection cn = ConexionBD.conectar();
            PreparedStatement ps = cn.prepareStatement(sql)) {
```

```

        ps.setString(1, expediente.getNumeroExpediente());
        ps.setInt(2, expediente.getIdArea());
        ps.setString(3, expediente.getAsunto());
        ps.setString(4, expediente.getResponsable());
        ps.setDate(5,
            java.sql.Date.valueOf(
                expediente.getFechaApertura()));
        ps.setString(6, expediente.getEstado());
        ps.setString(7, expediente.getUbicacion());

        ps.executeUpdate();

        return true;
    } catch (SQLException e) {

        e.printStackTrace();

        return false;
    }
}

@Override
public List<Expediente> listar() {

    List<Expediente> lista = new ArrayList<>();

    String sql = ""
        SELECT e.id_expediente,
               e.numero_expediente,
               e.id_area,
               a.nombre AS area,
               e.asunto,
               e.responsable,
               e.fecha_apertura,
               e.estado,
               e.ubicacion
        FROM expedientes e
        INNER JOIN areas a
            ON e.id_area = a.id_area
        ORDER BY e.id_expediente DESC
        """;

    try (Connection cn = ConexionBD.conectar();
        PreparedStatement ps = cn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {

            Expediente expediente = new Expediente();

            expediente.setId(
                rs.getInt("id_expediente"));

            expediente.setNumeroExpediente(
                rs.getString("numero_expediente"));

```

```

        expediente.setIdArea(
            rs.getInt("id_area"));

        expediente.setArea(
            rs.getString("area"));

        expediente.setAsunto(
            rs.getString("asunto"));

        expediente.setResponsable(
            rs.getString("responsable"));

        expediente.setFechaApertura(
            rs.getDate("fecha_apertura")
                .toLocalDate());

        expediente.setEstado(
            rs.getString("estado"));

        expediente.setUbicacion(
            rs.getString("ubicacion"));

        lista.add(expediente);
    }

} catch (SQLException e) {

    e.printStackTrace();
}

return lista;
}

@Override
public List<Expediente> buscar(String texto) {

    List<Expediente> lista = new ArrayList<>();

    String sql = ""
        SELECT e.id_expediente,
               e.numero_expediente,
               e.id_area,
               a.nombre AS area,
               e.asunto,
               e.responsable,
               e.fecha_apertura,
               e.estado,
               e.ubicacion
        FROM expedientes e
        INNER JOIN areas a
            ON e.id_area = a.id_area
        WHERE e.numero_expediente LIKE ?
            OR e.asunto LIKE ?
            OR a.nombre LIKE ?
        ORDER BY e.id_expediente DESC
        """;

```

```

try (Connection cn = ConexionBD.conectar();
    PreparedStatement ps = cn.prepareStatement(sql)) {

    String busqueda = "%" + texto + "%";

    ps.setString(1, busqueda);
    ps.setString(2, busqueda);
    ps.setString(3, busqueda);

    ResultSet rs = ps.executeQuery();

    while (rs.next()) {

        Expediente expediente = new Expediente();

        expediente.setId(
            rs.getInt("id_expediente"));

        expediente.setNumeroExpediente(
            rs.getString("numero_expediente"));

        expediente.setIdArea(
            rs.getInt("id_area"));

        expediente.setArea(
            rs.getString("area"));

        expediente.setAsunto(
            rs.getString("asunto"));

        expediente.setResponsable(
            rs.getString("responsable"));

        expediente.setFechaApertura(
            rs.getDate("fecha_apertura")
                .toLocaleDate());

        expediente.setEstado(
            rs.getString("estado"));

        expediente.setUbicacion(
            rs.getString("ubicacion"));

        lista.add(expediente);
    }

} catch (SQLException e) {

    e.printStackTrace();
}

return lista;
}

```

13. Crear la capa de servicio

Crear:

servicio/ExpedienteService.java

Código:

```
package archivo.municipal.servicio;

import archivo.municipal.dao.ExpedienteDAO;
import archivo.municipal.dao.ExpedienteDAOImpl;
import archivo.municipal.modelo.Expediente;

import java.util.List;

public class ExpedienteService {

    private final ExpedienteDAO expedienteDAO;

    public ExpedienteService() {
        expedienteDAO = new ExpedienteDAOImpl();
    }

    public boolean guardar(Expediente expediente) {

        if (expediente.getNumeroExpediente() == null ||
            expediente.getNumeroExpediente().trim().isEmpty()) {

            return false;
        }

        if (expediente.getAsunto() == null ||
            expediente.getAsunto().trim().isEmpty()) {

            return false;
        }

        return expedienteDAO.guardar(expediente);
    }

    public List<Expediente> listar() {

        return expedienteDAO.listar();
    }

    public List<Expediente> buscar(String texto) {

        return expedienteDAO.buscar(texto);
    }
}
```

La arquitectura queda:

INTERFAZ
↓
SERVICE
↓
DAO
↓
MYSQL

Esto será muy importante cuando posteriormente agreguemos documentos, usuarios, transferencias, etc.

14. Crear la ventana principal

La ventana tendrá aproximadamente:

Sistema Integral de Gestión Documental y Archivo Municipal									
Ayuntamiento	Usuario: admin								
Inicio	 Expedientes								
Expedientes	[+ Nuevo expediente] [Actualizar]								
Documentos	[Buscar...] 								
Consulta									
Administración	<table border="1"><thead><tr><th>Número</th><th>Área</th><th>Asunto</th><th>Responsable ...</th></tr></thead><tbody><tr><td>EXP...</td><td>...</td><td>...</td><td>...</td></tr></tbody></table>	Número	Área	Asunto	Responsable ...	EXP...	...	...	...
Número	Área	Asunto	Responsable ...						
EXP...	...	...	...						
Versión 0.1									

15. Colores de la interfaz

Utilizaremos una apariencia institucional:

```
private static final Color AZUL_OSCURO =  
    new Color(29, 55, 79);  
  
private static final Color AZUL_PRINCIPAL =  
    new Color(39, 111, 170);  
  
private static final Color AZUL_BOTON =  
    new Color(28, 110, 169);
```

```
private static final Color FONDO =  
    new Color(245, 248, 251);  
  
private static final Color BLANCO =  
    Color.WHITE;  
  
private static final Color TEXTO =  
    new Color(31, 51, 72);  
  
private static final Color BORDE =  
    new Color(220, 227, 234);
```

La intención es que se vea institucional y profesional, pero no excesivamente cargado.

16. Crear la ventana principal

Crear:

interfaz/MenuPrincipalFrame.java

El código debe encargarse de:

- Crear la ventana.
- Crear el menú lateral.
- Crear encabezado.
- Crear tabla.
- Cargar información de MySQL.
- Crear búsqueda.
- Crear botón Nuevo expediente.
- Crear botón Actualizar.

Código inicial:

```
package archivo.municipal.interfaz;  
  
import archivo.municipal.modelo.Expediente;  
import archivo.municipal.servicio.ExpedienteService;  
  
import javax.swing.*;  
import javax.swing.border.EmptyBorder;  
import javax.swing.table.DefaultTableCellRenderer;  
import javax.swing.table.DefaultTableModel;  
  
import java.awt.*;  
import java.awt.event.MouseAdapter;  
import java.awt.event.MouseEvent;  
import java.util.List;  
  
public class MenuPrincipalFrame extends JFrame {
```

```

private static final long serialVersionUID = 1L;

private final Color AZUL_OSCURO =
    new Color(29, 55, 79);

private final Color AZUL_PRINCIPAL =
    new Color(39, 111, 170);

private final Color AZUL_BOTON =
    new Color(28, 110, 169);

private final Color FONDO =
    new Color(245, 248, 251);

private final Color TEXTO =
    new Color(31, 51, 72);

private final Color BORDE =
    new Color(220, 227, 234);

private JTable tablaExpedientes;

private DefaultTableModel modeloTabla;

private JTextField txtBuscar;

private final ExpedienteService service =
    new ExpedienteService();

public MenuPrincipalFrame() {

    configurarVentana();

    crearInterfaz();

    cargarExpedientes();

}

private void configurarVentana() {

    setTitle(
        "Sistema Integral de Gestión Documental y Archivo Municipal"
    );

    setSize(1250, 750);

    setLocationRelativeTo(null);

    setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE
    );

    getContentPane().setBackground(FONDO);

}

private void crearInterfaz() {

```

```

        setLayout(new BorderLayout());

        crearMenuLateral();

        crearContenido();
    }

    private void crearMenuLateral() {

        JPanel menu = new JPanel();

        menu.setPreferredSize(
            new Dimension(230, 0));

        menu.setBackground(AZUL_OSCURO);

        menu.setLayout(
            new BoxLayout(
                menu,
                BoxLayout.Y_AXIS));

        JLabel lblTitulo =
            new JLabel(
                "<html><center>"
                + "<b>H. Ayuntamiento de</b><br>"
                + "Villa Aldama"
                + "</center></html>"
            );

        lblTitulo.setForeground(Color.WHITE);

        lblTitulo.setFont(
            new Font(
                "Segoe UI",
                Font.PLAIN,
                18));

        lblTitulo.setAlignmentX(
            Component.CENTER_ALIGNMENT);

        lblTitulo.setBorder(
            new EmptyBorder(
                30, 10, 30, 10));

        menu.add(lblTitulo);

        menu.add(crearBotonMenu("△ Inicio"));

        JButton btnExpedientes =
            crearBotonMenu("▣ Expedientes");

        btnExpedientes.setBackground(
            AZUL_PRINCIPAL);

        menu.add(btnExpedientes);
    }

```

```

        menu.add(
            crearBotonMenu("📁 Documentos"));

        menu.add(
            crearBotonMenu("🔍 Consulta"));

        menu.add(
            crearBotonMenu("⚙️ Administración"));

        menu.add(Box.createVerticalGlue());

        JLabel version =
            new JLabel("Versión 0.1");

        version.setForeground(
            new Color(210, 220, 230));

        version.setBorder(
            new EmptyBorder(
                0, 20, 20, 0));

        version.setAlignmentX(
            Component.LEFT_ALIGNMENT);

        menu.add(version);

        add(menu, BorderLayout.WEST);
    }

    private JButton crearBotonMenu(String texto) {

        JButton boton =
            new JButton(texto);

        boton.setMaximumSize(
            new Dimension(
                Integer.MAX_VALUE,
                55));

        boton.setAlignmentX(
            Component.CENTER_ALIGNMENT);

        boton.setHorizontalAlignment(
            SwingConstants.LEFT);

        boton.setForeground(Color.WHITE);

        boton.setBackground(AZUL_OSCURO);

        boton.setFont(
            new Font(
                "Segoe UI",
                Font.PLAIN,
                15));

        boton.setBorder(

```

```

        new EmptyBorder(
            0, 25, 0, 10));

    boton.setFocusPainted(false);

    return boton;
}

private void crearContenido() {

    JPanel contenido =
        new JPanel(
            new BorderLayout());

    contenido.setBackground(FONDO);

    contenido.setBorder(
        new EmptyBorder(
            25, 25, 25, 25));

    contenido.add(
        crearEncabezado(),
        BorderLayout.NORTH);

    contenido.add(
        crearPanelExpedientes(),
        BorderLayout.CENTER);

    add(
        contenido,
        BorderLayout.CENTER);
}

private JPanel crearEncabezado() {

    JPanel panel =
        new JPanel(
            new BorderLayout());

    panel.setOpaque(false);

    JLabel titulo =
        new JLabel("■ Expedientes");

    titulo.setForeground(TEXT0);

    titulo.setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            28));

    JLabel usuario =
        new JLabel(
            "Usuario: admin ▼");

    usuario.setForeground(TEXT0);

```

```

        usuario.setFont(
            new Font(
                "Segoe UI",
                Font.PLAIN,
                14));

        panel.add(
            titulo,
            BorderLayout.WEST);

        panel.add(
            usuario,
            BorderLayout.EAST);

        panel.setBorder(
            new EmptyBorder(
                5, 10, 25, 10));

        return panel;
    }

    private JPanel crearPanelExpedientes() {

        JPanel panel =
            new JPanel(
                new BorderLayout(0, 15));

        panel.setBackground(Color.WHITE);

        panel.setBorder(
            BorderFactory.createLineBorder(
                BORDE));

        panel.add(
            crearBarraHerramientas(),
            BorderLayout.NORTH);

        crearTabla();

        panel.add(
            new JScrollPane(
                tablaExpedientes),
            BorderLayout.CENTER);

        return panel;
    }

    private JPanel crearBarraHerramientas() {

        JPanel barra =
            new JPanel(
                new BorderLayout(15, 0));

        barra.setBackground(Color.WHITE);

        barra.setBorder(

```

```

        new EmptyBorder(
            15, 15, 0, 15));

JPanel botones =
    new JPanel(
        new FlowLayout(
            FlowLayout.LEFT,
            10,
            0));

botones.setOpaque(false);

JButton btnNuevo =
    new JButton(
        "+ Nuevo expediente");

btnNuevo.setBackground(
    AZUL_BOTON);

btnNuevo.setForeground(Color.WHITE);

btnNuevo.setFont(
    new Font(
        "Segoe UI",
        Font.BOLD,
        14));

btnNuevo.setFocusPainted(false);

btnNuevo.setBorder(
    new EmptyBorder(
        12, 18, 12, 18));

btnNuevo.addActionListener(e -> {

    NuevoExpedienteDialog dialog =
        new NuevoExpedienteDialog(
            this);

    dialog.setVisible(true);

    cargarExpedientes();
});

JButton btnActualizar =
    new JButton(
        "🔄 Actualizar");

btnActualizar.setFont(
    new Font(
        "Segoe UI",
        Font.PLAIN,
        14));

btnActualizar.setFocusPainted(false);

btnActualizar.addActionListener(

```

```

        e -> cargarExpedientes());

    botones.add(btnNuevo);

    botones.add(btnActualizar);

    JPanel busqueda =
        new JPanel(
            new BorderLayout(5, 0));

    busqueda.setOpaque(false);

    txtBuscar =
        new JTextField();

    txtBuscar.setPreferredSize(
        new Dimension(
            280,
            38));

    txtBuscar.setToolTipText(
        "Buscar por número, asunto o área");

    JButton btnBuscar =
        new JButton("⏏");

    btnBuscar.setBackground(
        AZUL_BOTON);

    btnBuscar.setForeground(
        Color.WHITE);

    btnBuscar.setFocusPainted(false);

    btnBuscar.addActionListener(
        e -> buscar());

    busqueda.add(
        txtBuscar,
        BorderLayout.CENTER);

    busqueda.add(
        btnBuscar,
        BorderLayout.EAST);

    barra.add(
        botones,
        BorderLayout.WEST);

    barra.add(
        busqueda,
        BorderLayout.EAST);

    return barra;
}

private void crearTabla() {

```

```

String[] columnas = {
    "Número",
    "Área",
    "Asunto",
    "Responsable",
    "Fecha de apertura",
    "Estado",
    "Ubicación",
    "Acciones"
};

modeloTabla =
    new DefaultTableModel(
        columnas,
        0) {

        @Override
        public boolean isCellEditable(
            int row,
            int column) {

            return false;
        }
    };

tablaExpedientes =
    new JTable(modeloTabla);

tablaExpedientes.setRowHeight(45);

tablaExpedientes.setFont(
    new Font(
        "Segoe UI",
        Font.PLAIN,
        13));

tablaExpedientes.setForeground(
    TEXTO);

tablaExpedientes.setGridColor(
    BORDE);

tablaExpedientes.setSelectionBackground(
    new Color(225, 239, 250));

tablaExpedientes.setSelectionForeground(
    TEXTO);

tablaExpedientes.getTableHeader()
    .setFont(
        new Font(
            "Segoe UI",
            Font.BOLD,
            13));

tablaExpedientes.getTableHeader()

```

```

        .setForeground(TEXT0);

tablaExpedientes.getTableHeader()
    .setBackground(
        new Color(238, 243, 248));

tablaExpedientes.getTableHeader()
    .setPreferredSize(
        new Dimension(
            0,
            42));

DefaultTableCellRenderer centro =
    new DefaultTableCellRenderer();

centro.setHorizontalAlignment(
    SwingConstants.CENTER);

tablaExpedientes.getColumnModel()
    .getColumn(0)
    .setPreferredWidth(110);

tablaExpedientes.getColumnModel()
    .getColumn(1)
    .setPreferredWidth(120);

tablaExpedientes.getColumnModel()
    .getColumn(2)
    .setPreferredWidth(220);

tablaExpedientes.getColumnModel()
    .getColumn(5)
    .setCellRenderer(
        new EstadoRenderer());

tablaExpedientes.addMouseListener(
    new MouseAdapter() {

        @Override
        public void mouseClicked(
            MouseEvent e) {

            if (e.getClickCount() == 2) {

                int fila =
                    tablaExpedientes
                        .getSelectedRow();

                if (fila >= 0) {

                    mostrarDetalle(fila);
                }
            }
        }
    });
}

```

```

private void cargarExpedientes() {

    List<Expediente> lista =
        service.listar();

    llenarTabla(lista);
}

private void buscar() {

    String texto =
        txtBuscar.getText().trim();

    if (texto.isEmpty()) {

        cargarExpedientes();

        return;
    }

    List<Expediente> lista =
        service.buscar(texto);

    llenarTabla(lista);
}

private void llenarTabla(
    List<Expediente> lista) {

    modeloTabla.setRowCount(0);

    for (Expediente e : lista) {

        modeloTabla.addRow(
            new Object[] {
                e.getNumeroExpediente(),
                e.getArea(),
                e.getAsunto(),
                e.getResponsable(),
                e.getFechaApertura(),
                e.getEstado(),
                e.getUbicacion(),
                "Ver"
            }
        );
    }
}

private void mostrarDetalle(int fila) {

    JOptionPane.showMessageDialog(
        this,
        "Número: "
        + modeloTabla.getValueAt(fila, 0)
        + "\nÁrea: "
        + modeloTabla.getValueAt(fila, 1)
        + "\nAsunto: "
    );
}

```

```

        + modeloTabla.getValueAt(fila, 2)
        + "\nResponsable: "
        + modeloTabla.getValueAt(fila, 3)
        + "\nFecha: "
        + modeloTabla.getValueAt(fila, 4)
        + "\nEstado: "
        + modeloTabla.getValueAt(fila, 5)
        + "\nUbicación: "
        + modeloTabla.getValueAt(fila, 6),
        "Información del expediente",
        JOptionPane.INFORMATION_MESSAGE
    );
}

private static class EstadoRenderer
    extends DefaultTableCellRenderer {

    @Override
    public Component getTableCellRendererComponent(
        JTable table,
        Object value,
        boolean isSelected,
        boolean hasFocus,
        int row,
        int column) {

        JLabel label =
            (JLabel) super
                .getTableCellRendererComponent(
                    table,
                    value,
                    isSelected,
                    hasFocus,
                    row,
                    column);

        label.setHorizontalAlignment(
            SwingConstants.CENTER);

        label.setOpaque(true);

        label.setBorder(
            new EmptyBorder(
                5, 10, 5, 10));

        if ("Abierto".equals(value)) {

            label.setBackground(
                new Color(55, 180, 125));

            label.setForeground(Color.WHITE);

        } else if ("En proceso".equals(value)) {

            label.setBackground(
                new Color(245, 175, 30));

```

```

        label.setForeground(Color.WHITE);
    } else {
        label.setBackground(
            new Color(120, 140, 155));
        label.setForeground(Color.WHITE);
    }
    return label;
}
}
}

```

17. Crear el formulario "Nuevo expediente"

El botón:

+ Nuevo expediente

deberá abrir un formulario.

El formulario contendrá:

```

Nuevo expediente

Número de expediente
[ EXP-2026-004 ]

Área
[ Presidencia ▼ ]

Asunto
[ _____ ]

Responsable
[ Administrador del Sistema ]

Fecha de apertura
[ 08/10/2026 ]

Estado
[ Abierto ▼ ]

Ubicación
[ Archivo de trámite ]

[ Cancelar ] [ Guardar ]

```

Esto corresponde a los datos básicos que ya contempla la especificación del expediente, aunque campos archivísticos adicionales como fondo, sección, serie, subserie, soporte y clasificación de acceso se incorporarán posteriormente conforme avance el sistema.

Crear:

interfaz/NuevoExpedienteDialog.java

Código:

```
package archivo.municipal.interfaz;

import archivo.municipal.modelo.Expediente;
import archivo.municipal.servicio.ExpedienteService;

import javax.swing.*;
import java.awt.*;
import java.time.LocalDate;

public class NuevoExpedienteDialog
    extends JDialog {

    private JTextField txtNumero;
    private JTextField txtAsunto;
    private JTextField txtResponsable;
    private JTextField txtUbicacion;

    private JComboBox<String> cbArea;

    private JComboBox<String> cbEstado;

    private final ExpedienteService service =
        new ExpedienteService();

    public NuevoExpedienteDialog(
        JFrame padre) {

        super(
            padre,
            "Nuevo expediente",
            true);

        setSize(500, 500);

        setLocationRelativeTo(padre);

        crearInterfaz();
    }

    private void crearInterfaz() {

        JPanel panel =
            new JPanel();

        panel.setBackground(Color.WHITE);
```

```

panel.setBorder(
    BorderFactory.createEmptyBorder(
        25, 30, 25, 30));

panel.setLayout(
    new GridLayout(
        0,
        1,
        10,
        8));

JLabel titulo =
    new JLabel(
        "Nuevo expediente");

titulo.setFont(
    new Font(
        "Segoe UI",
        Font.BOLD,
        24));

titulo.setForeground(
    new Color(31, 51, 72));

panel.add(titulo);

panel.add(
    new JLabel(
        "Número de expediente"));

txtNumero =
    new JTextField();

panel.add(txtNumero);

panel.add(
    new JLabel("Área"));

cbArea =
    new JComboBox<>(
        new String[] {
            "Presidencia",
            "Tesorería",
            "Obras Públicas",
            "Desarrollo Social",
            "Contraloría"
        });

panel.add(cbArea);

panel.add(
    new JLabel("Asunto"));

txtAsunto =
    new JTextField();

```

```

panel.add(txtAsunto);

panel.add(
    new JLabel("Responsable"));

txtResponsable =
    new JTextField(
        "Administrador del Sistema");

panel.add(txtResponsable);

panel.add(
    new JLabel(
        "Fecha de apertura"));

JTextField txtFecha =
    new JTextField(
        LocalDate.now().toString());

txtFecha.setEditable(false);

panel.add(txtFecha);

panel.add(
    new JLabel("Estado"));

cbEstado =
    new JComboBox<>(
        new String[] {
            "Abierto",
            "En proceso",
            "Cerrado"
        });

panel.add(cbEstado);

panel.add(
    new JLabel("Ubicación"));

txtUbicacion =
    new JTextField(
        "Archivo de trámite");

panel.add(txtUbicacion);

JPanel botones =
    new JPanel(
        new FlowLayout(
            FlowLayout.RIGHT));

JButton cancelar =
    new JButton("Cancelar");

cancelar.addActionListener(
    e -> dispose());

JButton guardar =

```

```

        new JButton("Guardar");

guardar.setBackground(
    new Color(28, 110, 169));

guardar.setForeground(
    Color.WHITE);

guardar.setFocusPainted(false);

guardar.addActionListener(
    e -> guardar());

botones.add(cancelar);

botones.add(guardar);

panel.add(botones);

add(panel);
}

private void guardar() {

    String numero =
        txtNumero.getText().trim();

    String asunto =
        txtAsunto.getText().trim();

    String responsable =
        txtResponsable.getText().trim();

    String ubicacion =
        txtUbicacion.getText().trim();

    if (numero.isEmpty()
        || asunto.isEmpty()
        || responsable.isEmpty()
        || ubicacion.isEmpty()) {

        JOptionPane.showMessageDialog(
            this,
            "Complete todos los campos.",
            "Datos incompletos",
            JOptionPane.WARNING_MESSAGE);

        return;
    }

    int idArea =
        cbArea.getSelectedIndex() + 1;

    String estado =
        cbEstado
        .getSelectedItem()
        .toString();

```

```

        Expediente expediente =
            new Expediente(
                numero,
                idArea,
                asunto,
                responsable,
                LocalDate.now(),
                estado,
                ubicacion);

        boolean guardado =
            service.guardar(expediente);

        if (guardado) {

            JOptionPane.showMessageDialog(
                this,
                "El expediente fue registrado correctamente.",
                "Registro exitoso",
                JOptionPane.INFORMATION_MESSAGE);

            dispose();

        } else {

            JOptionPane.showMessageDialog(
                this,
                "No fue posible registrar el expediente.",
                "Error",
                JOptionPane.ERROR_MESSAGE);

        }

    }
}

```

18. Crear Main

Crear:

Main.java

Código:

```

package archivo.municipal;

import archivo.municipal.interfaz.MenuPrincipalFrame;

import javax.swing.SwingUtilities;

public class Main {

    public static void main(String[] args) {

```

```

        SwingUtilities.invokeLater(() -> {

            MenuPrincipalFrame ventana =
                new MenuPrincipalFrame();

            ventana.setVisible(true);

        });
    }
}

```

19. ¿Qué ocurrirá cuando ejecutemos?

Al ejecutar:

Main.java

aparecerá:

Sistema Integral de Gestión Documental...	
Ayuntamiento	Usuario: admin
Inicio	☒ Expedientes
Expedientes	+ Nuevo expediente
Documentos	🔄 Actualizar
Consulta	
Admin	
	Buscar...
	Número Área Asunto ...

Al presionar:

Nuevo expediente

aparecerá el formulario.

Después de guardar:

```

Nuevo expediente
    ↓
ExpedienteService
    ↓
ExpedienteDAO
    ↓
ExpedienteDAOImpl
    ↓

```

ConexionBD



MySQL

y el expediente aparecerá en la tabla.

20. Prueba obligatoria del equipo

El equipo debe demostrar exactamente esto.

Prueba 1

Abrir el programa.

Prueba 2

Verificar que aparezcan los expedientes existentes.

Prueba 3

Presionar:

+ Nuevo expediente

Prueba 4

Registrar:

Número:
EXP-2026-004

Área:
Presidencia

Asunto:
Solicitud de información municipal

Responsable:
Administrador del Sistema

Estado:
Abierto

Ubicación:
Archivo de trámite

Prueba 5

Presionar:

Guardar

Prueba 6

Verificar:

El expediente fue registrado correctamente.

Prueba 7

Presionar:

Actualizar

Prueba 8

Verificar que:

EXP-2026-004

aparezca en la tabla.

Prueba 9

Cerrar el programa.

Prueba 10

Volverlo a abrir.

Prueba 11

Verificar que:

EXP-2026-004

continúe existiendo.

Esto último es fundamental porque demuestra que **no solamente lo guardaron en memoria: existe persistencia en la base de datos.**

Prueba 12

Utilizar el buscador:

Solicitud de información

y verificar que aparezca el expediente correspondiente.

21. ¿Qué demuestra esto de Programación Orientada a Objetos?

El proyecto puede justificar:

Encapsulamiento

La clase:

`Expediente`

contiene atributos privados:

```
private String numeroExpediente;  
private String asunto;  
private String responsable;
```

y métodos públicos:

```
getNumeroExpediente()  
setNumeroExpediente()
```

Abstracción

La interfaz:

`ExpedienteDAO`

define qué operaciones debe proporcionar el DAO.

Interfaces

```
public interface ExpedienteDAO
```

Implementación

```
public class ExpedienteDAOImpl  
    implements ExpedienteDAO
```

Separación de responsabilidades

Modelo
DAO
Servicio
Interfaz
Conexión

Persistencia

MySQL + JDBC.

Interfaz gráfica

Java Swing.

Flujo funcional

Registrar → Persistir → Consultar

22. ¿Qué NO debemos programar todavía?

Para evitar que el equipo intente terminar todo el proyecto de golpe, en este incremento **no es necesario desarrollar todavía:**

- Transferencias.
- Archivo de concentración.
- Archivo histórico.
- Préstamos.
- Valoración.
- Catálogo de disposición documental.
- Inventarios completos.
- Auditoría avanzada.
- Reportes.
- Firma electrónica.
- Aplicación móvil.
- Portal web.
- Integraciones externas.

La propia propuesta establece que esas capacidades se incorporarán progresivamente en versiones posteriores.

Tampoco debemos inventar desde ahora la clasificación archivística definitiva. El documento especifica que Fondo → Sección → Serie → Subserie → Expediente → Documento deberá configurarse con base en los instrumentos archivísticos autorizados y no ser determinado unilateralmente por los desarrolladores.

23. ¿Y los documentos PDF?

En este primer incremento podemos dejar preparado el botón:

```
Documentos
```

pero no necesitamos terminar todavía toda la carga de PDF.

La especificación sí contempla que el sistema pueda asociar documentos electrónicos a los registros archivísticos correspondientes, pero esa función puede integrarse progresivamente.

Posteriormente podremos crear:

```
Documento.java  
DocumentoDAO.java  
DocumentoDAOImpl.java  
DocumentoService.java
```

y una pantalla:

```
Nuevo documento  
  ↓  
Seleccionar PDF  
  ↓  
Datos del documento  
  ↓  
Asociar expediente  
  ↓  
Guardar
```

24. Organización de Git

Los cuatro integrantes deberán trabajar sobre el mismo repositorio.

Crear ramas:

```
main  
desarrollo  
bd  
poo  
interfaz  
integracion
```

Por ejemplo:

Integrante 1

bd

Commit:

Creación de base de datos inicial

Integrante 2

poo

Commit:

Implementación de modelo Expediente y DAO

Integrante 3

interfaz

Commit:

Diseño inicial de interfaz de expedientes

Integrante 4

integracion

Commit:

Integración interfaz con persistencia

Finalmente:

main

debe contener la versión funcional.

25. Evidencias que deben guardar

Para demostrar el trabajo académico, recomiendo tomar capturas de:

Evidencia 1

Estructura del proyecto.

Evidencia 2

Base de datos en MySQL.

Evidencia 3

Clase `Expediente`.

Evidencia 4

Interfaz `ExpedienteDAO`.

Evidencia 5

`ExpedienteDAOImpl`.

Evidencia 6

Interfaz principal.

Evidencia 7

Formulario Nuevo expediente.

Evidencia 8

Expediente registrado.

Evidencia 9

Expediente almacenado en MySQL.

Evidencia 10

Búsqueda funcionando.

Evidencia 11

Historial de commits de Git.

Evidencia 12

Programa ejecutándose.

26. Repartición final de responsabilidades

Integrante	Responsabilidad	Principales archivos
1	BD y conexión	ConexionBD, SQL
2	POO y persistencia	Expediente, DAO, DAOImpl, Service
3	Interfaz	MenuPrincipalFrame, NuevoExpedienteDialog
4	Integración y pruebas	Main, pruebas, Git, evidencias

Pero hay una regla importante:

Los cuatro deben conocer cómo funciona el sistema completo.

No conviene que una persona diga:

"Yo hice la base de datos y no sé cómo funciona el código."

Para la presentación, cualquiera debería poder explicar:

```
Interfaz
  ↓
Servicio
  ↓
DAO
  ↓
Conexión
  ↓
Base de datos
```

27. Resultado final del Incremento 1

Cuando terminen, podrán presentar:

"Se desarrolló un primer incremento funcional del Sistema Integral de Gestión Documental y Archivo Municipal para el H. Ayuntamiento de Villa Aldama, Veracruz. El incremento implementa una arquitectura inicial orientada a objetos, una base de datos relacional, clases modelo, una interfaz DAO, una capa de servicio y una interfaz gráfica en Java Swing. El flujo principal permite registrar un expediente, almacenarlo de manera persistente en MySQL, consultar los registros existentes y realizar búsquedas básicas."

Esto está alineado con el planteamiento del proyecto: construir progresivamente el sistema, conservar la estructura de información y permitir que el prototipo evolucione posteriormente hacia las funciones documentales y archivísticas completas.

28. El camino posterior

Una vez que este incremento funcione, **no vamos a desecharlo**.

La evolución sería:

INCREMENTO 1
Registro y consulta de expedientes

↓

INCREMENTO 2
Usuarios + login + roles

↓

INCREMENTO 3
Documentos + PDF

↓

INCREMENTO 4
Clasificación archivística

↓

INCREMENTO 5
Correspondencia

↓

INCREMENTO 6
Transferencias

↓

INCREMENTO 7
Préstamos y control físico

↓

INCREMENTO 8
Auditoría y seguridad

↓

INCREMENTO 9
Reportes

↓

VERSIÓN 1.0
Prototipo completo

↓

VERSIÓN 2.0
Sistema Integral Municipal

La propuesta original contempla precisamente una evolución de versiones desde el prototipo inicial hasta un sistema integral, en lugar de intentar construir todas las funciones simultáneamente.